

TALLINN UNIVERSITY OF TECHNOLOGY
School of Information Technologies

Ivan Kuznetsov 233767IVSB

LLM-Enhanced SAST: Reducing False Positives in Detection of Vulnerabilities in Source Code

Bachelor's thesis

Supervisor: Dr. Toomas Lepikult
PhD

Tallinn 2026

TALLINNA TEHNIKAÜLIKOOL
Infotehnoloogia teaduskond

Ivan Kuznetsov 233767IVSB

**LLM-i abil täiustatud SAST: valepositiivsete
juhtude vähendamine lähteteksti haavatavuste
tuvastamisel**

Bakalaureusetöö

Juhendaja: Dr. Toomas Lepikult
PhD

Tallinn 2026

Author's declaration of originality

I hereby certify that I am the sole author of this thesis. All the used materials, references to the literature and the work of others have been referred to. This thesis has not been presented for examination anywhere else.

Author: Ivan Kuznetsov

20.04.2026

Abstract

Static Application Security Testing (SAST) tools are foundational to the Secure Software Development Life Cycle (S-SDLC) but frequently generate high volumes of false-positive alerts, leading to critical operational "alert fatigue." While Large Language Models (LLMs) can effectively filter this noise, cloud-based solutions introduce severe data sovereignty risks, and local safety-aligned Small Language Models (SLMs) often suffer from "Defensive Refusal Bias," erroneously rejecting legitimate security analysis requests. This thesis investigates the efficacy of local, privacy-preserving SLMs in SAST alert triage and evaluates the empirical impact of removing safety alignment (abliteration) on defensive performance.

Using a structured, automated pipeline, 1,975 deterministically labeled Java files from the OWASP Benchmark are analyzed. The performance of a local 4B-parameter SLM (Qwen) is compared against cloud-based APIs (GPT-5 Nano, Claude Opus) across standard and high-complexity edge cases. The methodology involves extracting semantic method-level context to overcome token limits and enforcing a strict JSON schema for standardized Tier-2 security analysis.

The results demonstrate that safety alignment acts as a structural flaw in defensive workflows, artificially suppressing vulnerability detection. Applying an abliterated local SLM doubles the detection recall (+0.35) while maintaining high precision and completely eliminating safety-induced denial-of-service. However, local SLMs exhibit a rigid, hardware-bound "reasoning ceiling," failing to resolve intricate code conflicts that Tier-1 frontier models successfully process.

The thesis concludes by proposing a Hybrid DevSecOps Architecture: utilizing local, abliterated SLMs for zero-cost, privacy-compliant baseline noise reduction, and escalating only high-complexity edge cases to frontier cloud APIs.

The thesis is in English and contains 42 pages of text, 6 chapters, 10 figures, 3 tables.

List of abbreviations and terms

API	Application Programming Interface
CWE	Common Weakness Enumeration
Edge AI	Local AI deployment on consumer-grade hardware
FN	False Negative
FP	False Positive
GGUF	GPT-Generated Unified Format (quantized model format)
JSON	JavaScript Object Notation
LLM	Large Language Model
SAST	Static Application Security Testing
SLM	Small Language Model
TN	True Negative
TP	True Positive

Table of Contents

List of figures.....	9
List of tables	10
1 Introduction	11
1.1 Background and relevance.....	11
1.2 Research problem and objectives	12
1.3 Scope.....	13
2 Theoretical Background and Related Work	14
2.1 Static Application Security Testing (SAST) and Operational Risk.....	14
2.1.1 Industry perspective: Tools and Sovereignty	15
2.1.2 Academic studies: Quantifying Precision	15
2.2 Theoretical Frameworks and Research Gap	18
2.2.1 Theoretical Framing	18
2.2.2 Defensive Refusal Bias	18
2.2.3 Research Gap and Summary	18
3 Methodology and Pipeline Architecture.....	20
3.1 Methodology Overview	20
3.1.1 Research Goal and Operational Objectives.....	20
3.1.2 Three-Stage Research Workflow	21
3.1.3 Repository Structure and Implementation	24
3.2 Experimental Environment and Dataset	25
3.2.1 Hardware and Inference Stack	25
3.2.2 Evaluation Dataset: OWASP Benchmark v1.2	25
3.3 Pipeline Implementation	26
3.3.1 Initial Analysis and Automated Classification.....	26

3.3.2 Semantic Java Parsing and Context Preparation	28
3.3.3 Orchestration and Incident Response Framework	32
3.4 Evaluation Methodology.....	34
3.4.1 Analytical Variables and Formulas	34
3.4.2 Comparative Structure	35
3.4.3 Strategic Sampling and Conflict-Based Rationalization.....	35
3.5 Ethical and Privacy Considerations	36
3.5.1 Data Sovereignty and Compliance.....	37
3.5.2 Data Minimization and Ephemeral Processing	37
3.5.3 Local SLMs as Security Countermeasures	37
4 Results of the study.....	39
4.1 Dataset overview.....	39
4.1.1 Global Baseline Configuration.....	39
4.1.2 Conflict-Based Subset Justification	40
4.2 Global Quantitative Analysis (RQ1).....	40
4.3 The Impact of Abliteration (RQ2)	42
4.4 Frontier Model Comparison (RQ3)	43
4.5 Interpretations of Results	45
4.5.1 Structural Flaws and the Necessity of Abliteration.....	45
4.5.2 Parameter Count as a Rigid Reasoning Ceiling	46
4.5.3 Final Recommendation: Hybrid DevSecOps Architecture	46
5 Discussion.....	47
5.1 Theoretical implications	47
5.2 Practical implications.....	48
5.3 Implementing a Hybrid DevSecOps Architecture	49

6 Conclusion	51
6.1 Summary of findings	51
6.2 Limitations	52
6.3 Future directions	52
References	54
Appendix 1 – Non-exclusive licence for reproduction and publication of a graduation thesis	56
Appendix 2 – The Tier-2 Analyst System Prompt and JSON Schema	57
Appendix 3 – Semantic Java Context Parser Logic	59
Appendix 4 – Example of LLM Triage Output	61
Appendix 5 – Statistical Significance Calculations.....	62

List of figures

Figure 1. Annual publications on LLM-assisted static analysis (2017-2025), based on the author's bibliometric analysis.....	16
Figure 2. Refusal rates of safety-aligned models across different defensive cybersecurity tasks [2].....	19
Figure 3. Architectural overview of the automated LLM-augmented SAST triage pipeline.	20
Figure 4. Three-stage research workflow illustrates the transition from initial data acquisition through semantic structuring to final LLM-based validation.	23
Figure 5. High-level structure of the automated SAST-LLM triage and analysis repository.	24
Figure 6. Logic flow of the automated classification script, demonstrating the sorting of findings into True Positive, False Positive, and False Negative directories based on ground-truth comparison.	28
Figure 7. Visualization of the semantic parsing process, showing the extraction of global headers, the execution of the brace-counting algorithm, and the application of windowing for long methods.	31
Figure 8. Architecture of the LLM orchestration engine, highlighting the integration of the Tier-2 Analyst and the enforcement of a strict JSON schema for standardized security reporting.....	33
Figure 9. Performance comparison.....	41
Figure 10. The Impact of Safety Alignment.....	43

List of tables

Table 1. Comparative overview of LLM-SAST studies.....	17
Table 2. Global Model Performance	40
Table 3. Frontier Evaluation of 200 Conflict Cases.	44

1 Introduction

The integration of automated security tools into software development has changed how vulnerabilities are identified. Alongside traditional testing methods, Large Language Models (LLMs) are increasingly being used to analyze code logic. This thesis examines how local Small Language Models (SLMs) can reduce the operational noise generated by Static Application Security Testing (SAST) and improve vulnerability triage.

1.1 Background and relevance

Static Application Security Testing (SAST) is a foundational component of the Software Development Life Cycle (SDLC). It allows organizations to scan source code for known security flaws without executing the program. However, SAST tools rely heavily on rigid pattern matching and technical representations, such as Abstract Syntax Trees. Because they often lack an understanding of the broader code context across different functions, they generate a high volume of inaccurate alerts.

From an operational perspective, this noise creates a significant risk known as "alert fatigue." Security teams spend excessive time manually verifying issues that are not actual vulnerabilities. This reduces the overall effectiveness of the testing process, as real threats can be easily overlooked among the false alarms. To handle this burden, the software industry is increasingly looking for semantic filtering methods that can automatically verify these alerts and understand the actual intent of the code.

While cloud-based Large Language Models have shown they can eliminate up to 98% of SAST false positives [1], their use presents a major compliance issue. Source code is highly sensitive intellectual property. Sending security telemetry to external cloud providers often violates data sovereignty, privacy requirements, and residency regulations. As a result, organizations are trying to shift toward local Small Language Models (SLMs) that can run entirely on internal hardware.

Despite the growing interest in local deployment, the academic and practical application of SLMs in cybersecurity faces specific technical barriers. Most existing studies focus on massive frontier models, leaving a gap regarding the performance of models under 5 billion parameters (Edge AI) on consumer-grade hardware. Furthermore, safety-aligned models often exhibit a "Defensive Refusal Bias" [2]. Because they are trained to reject malicious prompts, they frequently refuse to analyze legitimate vulnerable code, confusing defensive security tasks with harmful intent, leading to a safety-induced denial-of-service. The novelty of this thesis lies in addressing these specific hardware and alignment gaps. Unlike previous studies that rely on closed-source cloud APIs, this research evaluates hardware-constrained local models. It also investigates whether removing the safety alignment (ablation) from these models can restore their ability to perform security triage without losing analytical precision.

1.2 Research problem and objectives

The central research challenge addressed in this study is the high rate of false positives in automated security scanning and the technical barriers to filtering them locally.

Although existing research demonstrates that AI can filter SAST noise [1] [3], there is a lack of empirical data on how local, privacy-preserving models compare to cloud APIs. Additionally, the negative impact of built-in safety filters on defensive security tasks remains an unresolved operational obstacle for local deployment.

The study is guided by three research questions:

- **RQ1:** How do local Small Language Models (SLMs) compare to cloud-based APIs in filtering SAST false positives globally?
- **RQ2:** What is the empirical impact of removing safety alignment (ablation) on a model's defensive recall?
- **RQ3:** How do local SLMs perform against frontier models (Claude Opus) on high-complexity edge cases?

The main objective of this study is to quantify the performance gaps between local SLMs and cloud-based APIs in identifying false-positive security alerts. The research evaluates a structured dataset of Java files to measure precision, recall, and operational stability. The expected outcome is an empirically supported recommendation for deploying AI in security operations, specifically through a tiered hybrid architecture.

1.3 Scope

This study focuses exclusively on the automated triage of Static Application Security Testing (SAST) alerts using language models. The dataset consists of 1,975 deterministically labeled Java source files extracted from the OWASP Benchmark version 1.2. The evaluation is restricted to sub-5B parameter models running on consumer-grade hardware (6GB VRAM) and specific cloud-based APIs (GPT-5 Nano and Claude Opus 4.6). The research strictly evaluates the "Detection and Analysis" phase of incident response. Model training, fine-tuning, and the discovery of novel zero-day vulnerabilities fall outside the scope of this analysis. All processing is conducted locally to respect data sovereignty principles, and no proprietary code is exposed to external services or used for model training.

2 Theoretical Background and Related Work

This chapter provides a comprehensive overview of the technical and theoretical foundations required to understand the integration of machine learning into software security workflows. It begins by analyzing the mechanics of traditional Static Application Security Testing (SAST) and its inherent operational risks. The discussion then shifts to the emergence of Small Language Models (SLMs) and their specialized application in semantic code analysis. Finally, the chapter evaluates existing hybrid approaches and identifies the specific research gaps - such as hardware constraints and safety-alignment biases - which this thesis addresses.

2.1 Static Application Security Testing (SAST) and Operational Risk

Static Application Security Testing (SAST) is a foundational but frequently noisy layer within the Software Development Life Cycle (SDLC). It represents a critical automated control for identifying security flaws without program execution. Conceptually, it works by scanning source code to identify patterns that match known security weaknesses. To analyze code structure, tools rely on technical representations such as the Abstract Syntax Tree (AST) and the Control Flow Graph (CFG) [4] [5].

However, the practical effectiveness of SAST is often hindered by the high volume of inaccurate alerts, which contributes to organizational exposure by masking real vulnerabilities within "operational noise." This noise is primarily caused by two significant operational risks rather than simple technical flaws:

- **Path-insensitivity:** Occurs when tools utilize coarse-grained rules or regular expressions that fail to account for complex logic across method boundaries or object interactions.
- **The Reachability Gap:** Refers to the difficulty of proving whether a specific code path is actually executable, a challenge linked to the inherent undecidability of static analysis [5].

Empirical research indicates that many tools are not "vulnerability-sensitive" enough to handle real-world complexity, often reporting the same issues regardless of whether a patch has been applied. Consequently, the resulting "alert fatigue" reduces the overall defensive value of the testing process. These persistent limitations highlight a significant research need for semantic filtering methods that can better understand program context and reduce the burden of manual triage [4] [6].

2.1.1 Industry perspective: Tools and Sovereignty

Industrial practitioners frequently prioritize speed and integration, leading to the widespread deployment of lightweight scanners such as Semgrep. While these tools are efficient for scanning code during the commit process, their reliance on patterns often results in a high volume of inaccurate findings. These practitioner insights illustrate the operational realities of manual triage, providing evidence of defensive shortcomings that complement academic literature [3] [7].

To manage this burden, industry is increasingly shifting toward SLMs as a functional alternative to external cloud-based APIs. This transition is primarily driven by data sovereignty and compliance requirements. Security telemetry often contains highly sensitive information that organizations are prohibited from sending to external services due to residency obligations. By deploying SLMs on premises, enterprises can maintain full control over their sensitive data within their own infrastructure while reducing the high operational expenses associated with proprietary cloud services [1].

2.1.2 Academic studies: Quantifying Precision

To situate this research within the global academic landscape, a preliminary bibliometric analysis was conducted using Google Scholar. Relevant literature was identified using the exact phrase search query for "LLM Static Analysis". As shown in Figure 1, annual publication activity has grown exponentially in recent years. This explosive upward trend - from 847 results in 2017 to nearly 38,900 in 2025 - demonstrates that integrating Large Language Models into static analysis workflows is an increasingly critical research concern.

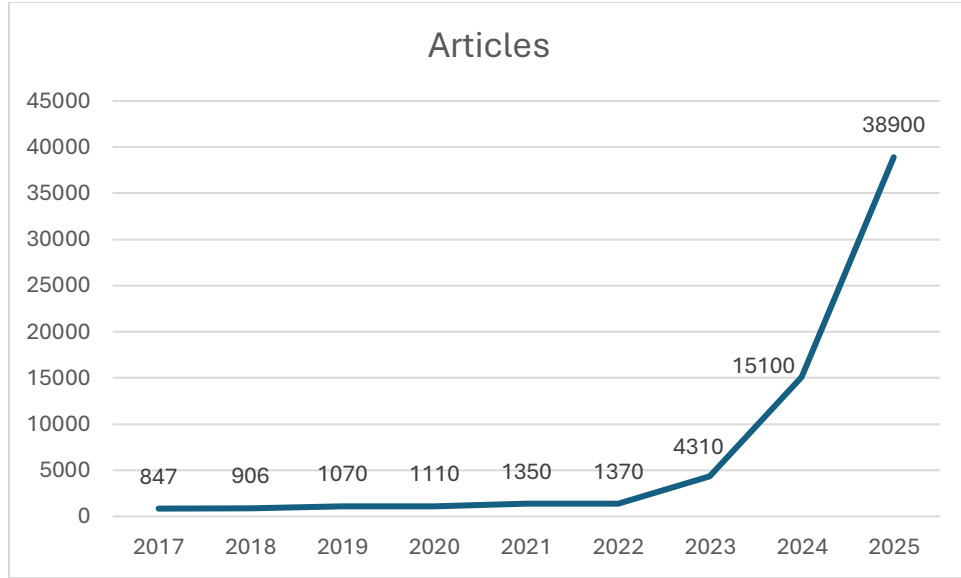


Figure 1. Annual publications on LLM-assisted static analysis (2017-2025), based on the author's bibliometric analysis.

Beyond publication volume, academic research has quantified the effectiveness of LLMs in serving as semantic filters for static analysis noise. Empirical evaluations in large-scale industrial environments demonstrate that hybrid LLM-SAST pipelines can eliminate 94% to 98% of false positives while maintaining high detection rates [1]. Furthermore, studies on agentic frameworks show that iterative reasoning can reduce initial false positive rates from over 92% to as low as 6.3% in optimal configurations [3].

A significant academic consensus suggests that the primary driver of these improvements is the inclusion of context. Researchers emphasize that models must go beyond isolated code snippets to perform accurate reasoning. Effective frameworks provide a comprehensive view of the "sink," specifically including its callers and callees. Without this deep contextual understanding, models are prone to reasoning errors and hallucinations, failing to resolve the complex logical dependencies that cause most false alarms [3]. Table 1 provides a comparative overview of recent literature on integrating LLMs into static analysis workflows, highlighting both their methodologies and the remaining research gaps.

Table 1. Comparative overview of LLM-SAST studies.

Study / Source	Methodology	Main Findings	Research Gap
Industrial Evaluation (Tencent) [1]	Empirical evaluation of LLM techniques on 433 real-world bug alarms (NPD, OOB, DBZ) from an enterprise-customized static analyzer [1].	Hybrid LLM-SAST techniques eliminated 94-98% of false positives while maintaining high recall [1].	Relies on closed-source cloud APIs, which conflicts with data sovereignty and privacy requirements for sensitive industrial code.
"Sifting the Noise" (Agentic) [3]	Comparative study of three agent frameworks (Aider, OpenHands, SWE-agent) using the OWASP Benchmark and Vul4J dataset [3].	Best configurations reduced initial false positive rates from 98.3% to 6.3% through cross-file semantic resolution [3].	High operational costs and token usage; predominantly utilize massive cloud-based frontier models rather than hardware-constrained local alternatives.
Cybersecurity-Expert SLM [8] [9]	Training specialized SLMs (4B-20B) using expert-curated reasoning traces and multi-step evidence grounding [8] [9].	SLMs matched or surpassed frontier models on specific threat intelligence tasks while remaining cost-effective [8] [9].	While supporting on-premises use, it fails to investigate the impact of defensive refusal bias or performance on Edge AI hardware under 5B parameters.
Defensive Refusal Bias [2]	Analysis of 2,390 prompts from the NCCDC competition to evaluate safety-induced denial of assistance in defensive workflows [2].	Aligned models refuse legitimate defensive requests at 2.72x the rate of neutral requests when security keywords like "exploit" are present [2].	Documents show the systematic refusal phenomenon but do not provide a technical solution or comparative triage accuracy for uncensored vs. aligned models.

This comparative overview highlights that while existing research demonstrates the potential of LLMs to reduce static analysis noise, a significant gap remains in delivering privacy-preserving solutions that function on local hardware without being hindered by alignment-induced refusals.

2.2 Theoretical Frameworks and Research Gap

This section outlines the conceptual framework that positions LLMs within security operations and identifies the technical barriers addressed by this study.

2.2.1 Theoretical Framing

This thesis adopts a triage-centric perspective, conceptualizing the Large Language Model as an automated "Tier-2 Analyst." To ensure industry relevance, this designated role is strictly structured according to the NIST SP 800-61 guidelines for computer security incident handling (the specific system prompt designed for this role is provided in Appendix 2).

Within this established framework, the model moves beyond basic code completion to execute the "Detection and Analysis" phase of incident response. By evaluating specific impact factors - such as functional impact, information risk, and recoverability effort - the pipeline utilizes standardized cybersecurity terminology. This approach avoids the creation of ad-hoc classification labels, ensuring that the automated triage process remains fully compatible with standard Security Operations Center (SOC) workflows [10].

2.2.2 Defensive Refusal Bias

A primary technical obstacle to this automated triage is *Defensive Refusal Bias*, a phenomenon where safety-aligned models deny assistance for legitimate security tasks. This results in a safety-induced denial-of-service for defenders because models rely on semantic proximity to harmful content rather than reasoning about intent or authorization.

As illustrated in Figure 2, recent empirical studies reveal that the most operationally critical security tasks experience the highest refusal rates. For instance, safety-aligned models refuse "System Hardening" and "Malware Analysis" requests 43.8% and 34.3% of the time, respectively, while neutral tasks like "Log Analysis" face zero refusals. Furthermore, explicit authorization signals often backfire, as models may interpret justifications as adversarial jailbreak attempts. This bias creates a significant operational barrier for local deployment, as the use of sensitive cybersecurity terminology is unavoidable [2].

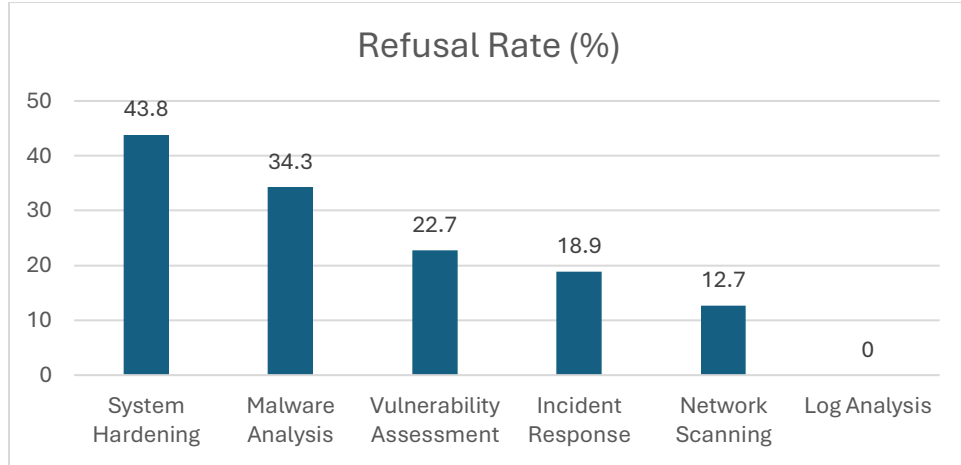


Figure 2. Refusal rates of safety-aligned models across different defensive cybersecurity tasks [2].

2.2.3 Research Gap and Summary

Despite the promise of SLMs for privacy-preserving detection, significant research gaps remain. While fine-tuned models with 350 million parameters can identify vulnerabilities, there is insufficient comparative data for sub-5B "Edge AI" models running on local hardware. Most existing studies rely on cloud-based frontier models that conflict with data sovereignty requirements [11].

Additionally, there is a lack of technical evaluation regarding how uncensored "abliterated" weights affect triage performance on constrained hardware. This thesis addresses these gaps by evaluating if removing safety alignment can restore defensive utility without sacrificing precision [2].

The reviewed literature establishes that while LLMs can significantly reduce SAST noise, safety-alignment and hardware constraints remain unresolved obstacles. Through this approach, the thesis aims to contribute both a technical solution for Edge AI deployment and an evaluation of uncensored weights in professional security triage.

3 Methodology and Pipeline Architecture

This chapter describes the research methodology, the technical architecture of the pipeline, and the evaluation strategy used in this study. It provides a detailed look at how the experiments were organized to test the effectiveness of different language models in identifying software vulnerabilities. An architectural overview of this complete pipeline is illustrated in Figure 3.

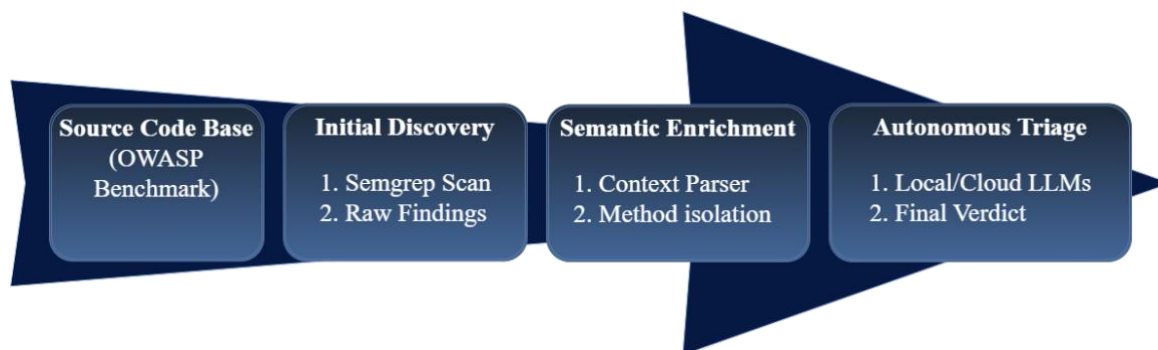


Figure 3. Architectural overview of the automated LLM-augmented SAST triage pipeline.

3.1 Methodology Overview

This section describes the research design used to evaluate the effectiveness of Large Language Models (LLMs) in improving software security scans. The study focuses on how local and cloud-based models can identify errors in static analysis results. By following a systematic process, the research ensures that the comparison between different model types is fair, repeatable, and grounded in measurable data.

3.1.1 Research Goal and Operational Objectives

The main objective of this study is to compare the performance of local Small Language Models (SLMs) against cloud-based APIs in identifying false-positive security alerts.

To achieve this objective, the research is structured into the following operational sub-goals:

1. A baseline of initial security alerts is generated by performing scans on the OWASP Benchmark using Semgrep.
2. Initial alerts are validated against deterministic ground truth to differentiate between true positives and false positives.
3. Files are categorized and migrated into structured directories based on scan results to enable targeted analysis.
4. Semantic context is extracted by isolating specific Java methods related to each alert to provide models with necessary logic.
5. Automated audits are conducted using local SLMs and cloud-based models to evaluate the validity of each security alert.
6. Performance gaps are quantified by comparing precision, recall, and hardware resource usage between local and cloud-based execution.

These methodological choices support both technical accuracy and reproducibility. Semgrep and the OWASP Benchmark were selected for their standardized nature; the use of local SLMs addresses data privacy concerns; and statistical comparisons of F1-scores enable a direct assessment of model reliability.

3.1.2 Three-Stage Research Workflow

The experimental process is divided into three distinct stages, as outlined in Figure 4. This workflow ensures that data flows logically from raw source code to final performance metrics

Stage 1 - Acquisition: The focus of this stage is data collection. The research uses the OWASP Benchmark, a large-scale Java project designed to test the accuracy of security tools. By running Semgrep, the system acquires a raw dataset of security findings. The logic here is to start with a "noisy" dataset that contains many false alerts, mirroring the real-world challenges faced by software developers.

Stage 2 - Structuring: Once the raw alerts are collected, they must be prepared for the language models. This stage involves transforming flat JSON alerts into rich, method-level context. Custom Python scripts locate the exact line of code flagged by the scanner and extract the surrounding function. This step is necessary because LLMs perform better when they understand the intent and logic of the code rather than just seeing a single line.

Stage 3 - Validation: In the final stage, the models act as automated security analysts. The system sends structured data to each model and records its verdict. These verdicts are then validated against the ground truth. This stage provides the evidence needed to conclude which models are most reliable for reducing the manual workload of security teams.

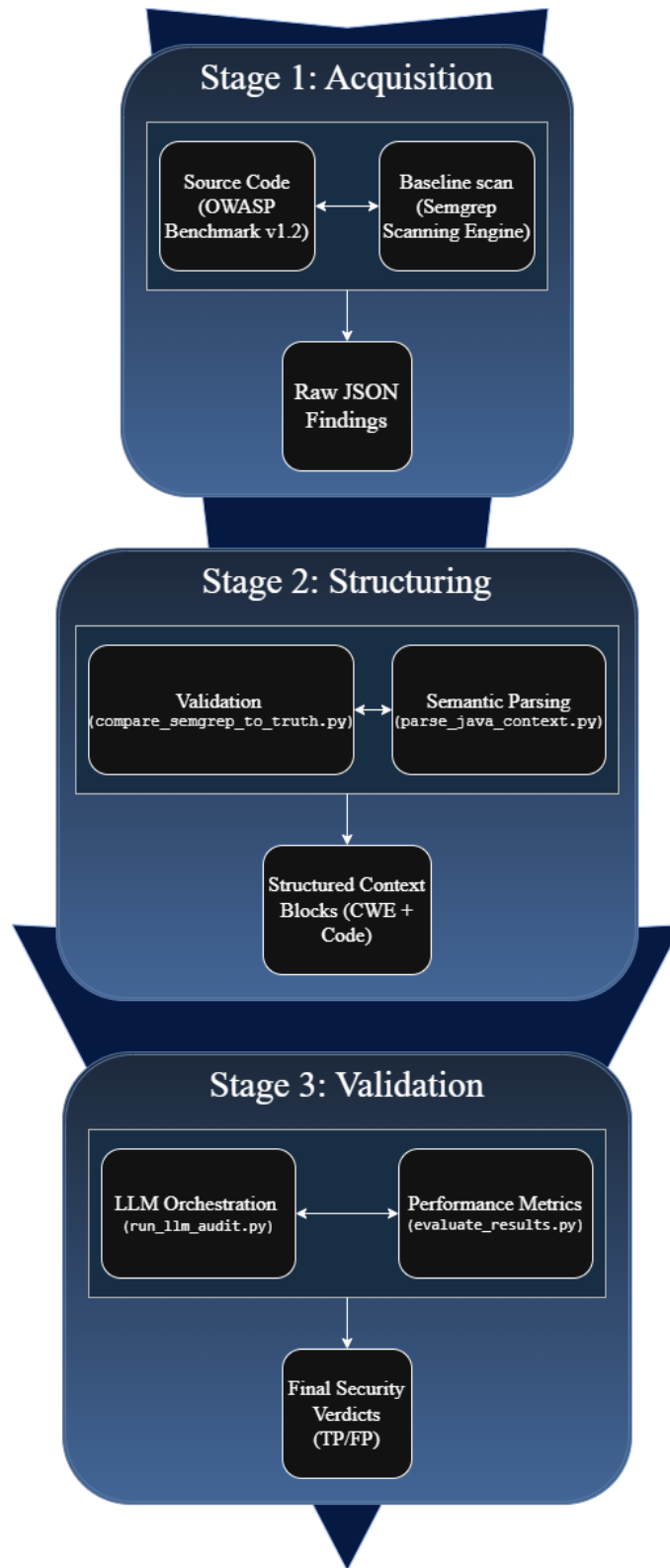


Figure 4. Three-stage research workflow illustrates the transition from initial data acquisition through semantic structuring to final LLM-based validation.

3.1.3 Repository Structure and Implementation

The research is implemented through a modular Python-based pipeline. Each script handles a specific part of the workflow, allowing for individual testing and debugging.

To support full reproducibility and transparency of the experimental pipeline, the complete source code, including the semantic Java parser and LLM orchestration scripts, is publicly accessible in the project's GitHub repository: <https://github.com/iznetsss/thesis-project-ivsb>

The high-level structure of this repository is detailed in Figure 5 and organized as follows:

```
project/
├── data/                                # Ground truth and source code repository
│   ├── benchmark_source/                # Java source files from OWASP Benchmark
│   │   ├── research_files/              # Files categorized for LLM analysis
│   │   │   ├── blind_test/              # Contains False Negatives (missed by SAST)
│   │   │   └── sast_alerts/             # Contains TP and FP from SAST
│   │   └── corpus_storage/              # Archive for non-vulnerable (TN) files
│   └── expectedresults_full.csv          # OWASP Benchmark ground truth
├── results/                             # Outputs from scanners and language models
│   ├── models_results/                  # Raw JSON verdicts from all evaluated LLMs
│   ├── initial_scan.json                 # Baseline results from the Semgrep engine
│   ├── llm_prompts.json                  # Final dataset with extracted semantic context
│   ├── opus_queue_200.json               # Targeted subset for high-cost model testing
│   └── semgrep_accuracy_report.csv        # Baseline scanner comparison
├── scripts/                             # The automated research pipeline (1-8)
│   ├── 1_run_semgrep.md                  # Instructions for the initial scan
│   ├── 2_compare_semgrep_to_truth.py     # Scanner finding validation
│   ├── 3_calculate_semgrep_accuracy.py    # Baseline metrics generation
│   ├── 4_sort_research_data.py           # Automated file migration logic
│   ├── 5_parse_java_context.py           # Semantic method-level extraction
│   ├── 6_select_opus_200.py              # Logic for conflict-based sampling
│   ├── 7_run_llm_audit.py                # Main LLM orchestration engine
│   ├── 8_evaluate_results.py             # Final metrics calculations
│   └── chapter4-statistical_tests.py      # Stat significance validation
├── .env                                 # API keys
├── java_parsing_structure.md             # Technical specs for the context
├── parser
└── requirements.txt                       # Project dependencies
```

Figure 5. High-level structure of the automated SAST-LLM triage and analysis repository.

Use of Generative AI Disclosure. The use of Generative AI in this study is limited to the automated classification of security alerts and the structural refinement of the thesis text. All experimental results produced by the models are recorded as primary research data. No

proprietary or sensitive code was used to train external models, and all AI-generated outputs were reviewed by the researcher to ensure technical accuracy and academic integrity.

3.2 Experimental Environment and Dataset

This section details the hardware and software configuration used for the experiments, as well as the characteristics of the dataset selected for evaluation.

3.2.1 Hardware and Inference Stack

The experiments were conducted on a local workstation equipped with an Intel i7-12650H processor and 64GB of DDR5 RAM. The primary technical constraint in this environment was the NVIDIA RTX 3060 graphics card, which features 6GB of Video RAM (VRAM). Because VRAM determines the size of the model that can be loaded into the GPU memory, it was necessary to select models that could function within these limits [9] [11].

To address these hardware boundaries, the research utilized 4-bit quantized models in the GGUF format. Quantization is a technique that reduces the memory requirements of a model by lowering the precision of its internal weights. By using 4-bit quantization, the model size is significantly reduced, allowing the RTX 3060 to process security tasks that would otherwise require enterprise-grade hardware with higher VRAM [9].

For model hosting, the LM Studio platform was used. This software creates a local server that mimics the OpenAI API structure. This setup allows the Python-based pipeline to communicate with the local models using standard protocols. By hosting the models locally, all code analysis is performed on the internal hardware, ensuring that no data is sent to external cloud providers.

3.2.2 Evaluation Dataset: OWASP Benchmark v1.2

The performance of the models was tested using OWASP Benchmark version 1.2. This dataset is a recognized standard for evaluating the accuracy of automated security tools [4]. For this study, a subset of 1,975 Java files was selected. Java was chosen strictly because the OWASP Benchmark version 1.2 is the only comprehensive open-source project that provides

the deterministic, developer-verified ground truth required for scientific validation. Every file in the dataset is pre-labeled as either a "True Positive" (containing a real vulnerability) or a "False Positive" (containing safe code that appears suspicious). This clear labeling is essential for scientific evaluation. It allows the research to calculate specific performance metrics, including:

- **Precision:** The percentage of security alerts that were actually real vulnerabilities.
- **Recall:** The ability of the model to find all existing vulnerabilities in the dataset.
- **F1-Score:** The harmonic mean of precision and recall, providing a single score for overall accuracy.

By using a standardized benchmark with known results, the study provides an objective measurement of how well local models can distinguish between real security threats and harmless code patterns.

3.3 Pipeline Implementation

The technical execution of the research is handled by an automated pipeline. This pipeline manages the transition from raw source code to structured data and, finally, to model-generated security verdicts. The system is designed to remove human bias by using standardized scripts for every step of the process.

3.3.1 Initial Analysis and Automated Classification

The first stage of the pipeline establishes a performance baseline using the Semgrep engine. This tool scans the Java source files within the OWASP Benchmark to identify potential security flaws. Once the scan is complete, the results are exported as a JSON report.

To evaluate the accuracy of these initial findings, the pipeline uses a script named `compare_semgrep_to_truth.py`. This script performs a cross-reference between the Semgrep report and the `expectedresults_full.csv` file, which serves as the "ground truth." As shown in Figure 6, the logic follows a strict classification path:

- **True Positives (TP):** If the scanner flags a line that the ground truth confirms as vulnerable, the file is moved to the TP directory.
- **False Positives (FP):** If the scanner flags a line that is marked as "safe" in the ground truth, the file is moved to the FP directory.
- **False Negatives (FN):** Files that contain known vulnerabilities but were missed by the scanner are moved to a separate "blind test" directory.

This physical separation of files is a design choice that allows the research to isolate specific failure points of the SAST tool. By grouping False Positives together, the pipeline can specifically test if the language models can identify the logical errors made by the initial scanner.



Figure 6. Logic flow of the automated classification script, demonstrating the sorting of findings into True Positive, False Positive, and False Negative directories based on ground-truth comparison.

3.3.2 Semantic Java Parsing and Context Preparation

The effective use of language models in security analysis is limited by the token window constraint. To ensure the model receives sufficient context without exceeding memory limits, a semantic Java parsing process is implemented via the `parse_java_context.py` script. This process transforms raw SAST alerts into structured context blocks through three distinct technical phases: global header extraction, method boundary discovery, and semantic windowing (the complete algorithmic logic for this extraction process is detailed in Appendix 3).

Global Header Extraction. The first phase involves identifying the environmental context of the source file. The parser scans the initial lines of the Java file (typically the first 50 lines) to extract the package declaration and all import statements. This step is critical for identifying the external libraries and security frameworks utilized by the application. By providing these headers, the model can determine if the code utilizes standardized security controls or vulnerable legacy libraries, which is essential for accurate data-flow analysis.

Method Boundary Discovery. Method boundaries are identified through a Brace-Counting Algorithm. This approach is utilized because Java methods vary significantly in length, making fixed-line windows unreliable for capturing complete logical units. As illustrated in Figure 7, the process follows a deterministic logic:

1. An upward search is performed from the line flagged by the scanner to locate the method signature (e.g., `public void handle()`).
2. A counter is initialized at zero.
3. The algorithm scans downward, incrementing the counter for every open brace `{` and decrementing it for every closing brace `}`.
4. The method boundary is defined when the counter returns to zero after the initial open brace.

The Windowing Technique For methods exceeding 100 lines, a windowing technique is applied to maintain analytical focus while preserving the model’s reasoning performance.

- The method signature and the first 10 lines of the body are preserved to provide the model with entry logic and variable initializations.
- A 100-line window is centered on the specific line where the vulnerability was detected.
- Redundant code segments between the initial lines and the centered window are replaced with a `// ... [code skipped] ... comment`. This ensures that the most

relevant security logic remains intact while staying within the model's efficient processing range.

Code Normalization and Tagging. Finally, the extracted code is normalized to assist model interpretation and cross-referencing. Every line is prefixed with a line-number (e.g., 104:), and a specific marker (>>) is injected at the exact location of the SAST alert. This normalization allows the model to reference specific code segments in its technical justification, increasing the transparency of the automated audit.

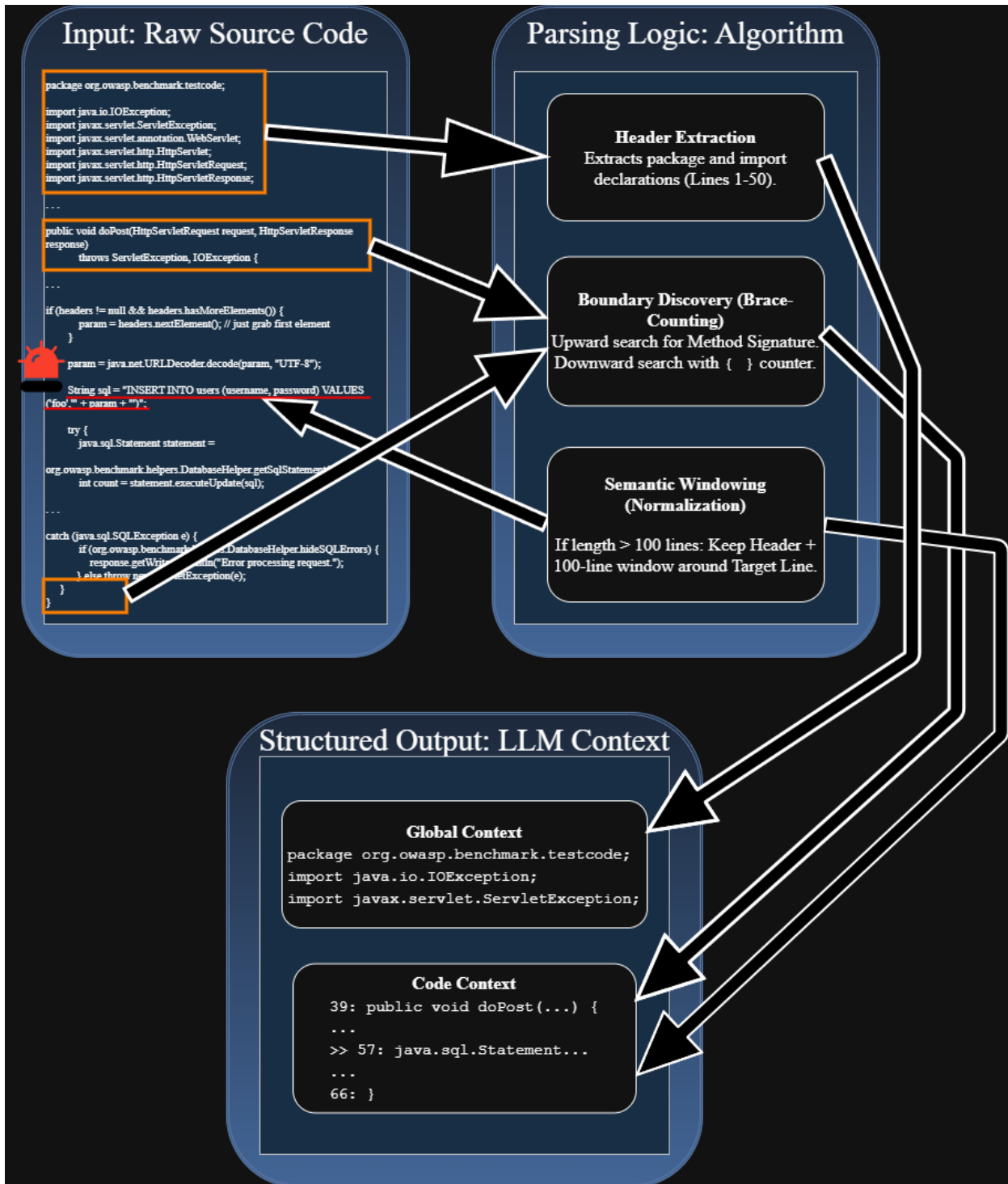


Figure 7. Visualization of the semantic parsing process, showing the extraction of global headers, the execution of the brace-counting algorithm, and the application of windowing for long methods.

3.3.3 Orchestration and Incident Response Framework

The final stage of the pipeline is managed by `run_llm_audit.py`. The design of this script is influenced by the NIST SP 800-61 standard for computer security incident handling. Specifically, the script mimics the "Detection and Analysis" phase of the incident life cycle.

The implementation follows three principles derived from the NIST standard:

1. **Objective Analysis:** The model is given a "Tier-2 Analyst" role and instructed to be skeptical. It does not simply accept the SAST alert as true; it must analyze the code logic to verify the findings.
2. **Standardized Reporting:** Just as NIST requires consistent documentation, the script enforces a strict JSON schema. The model must provide a verdict, a confidence score, and a technical justification (a representative example of this structured JSON output is provided in Appendix 4). This allows for automated comparisons between different models.
3. **Triage Prioritization:** The script manages two distinct flows. The "SAST Triage" flow focuses on confirming or dismissing existing alerts (addressing False Positives). The "Blind Test" flow mimics a proactive search for hidden threats (addressing False Negatives).

By linking the script's logic to an established framework like NIST SP 800-61, the research ensures that the automated auditing process reflects professional security operations. The complete architecture of this orchestration engine, including the strict JSON schema enforcement, is presented in Figure 8.

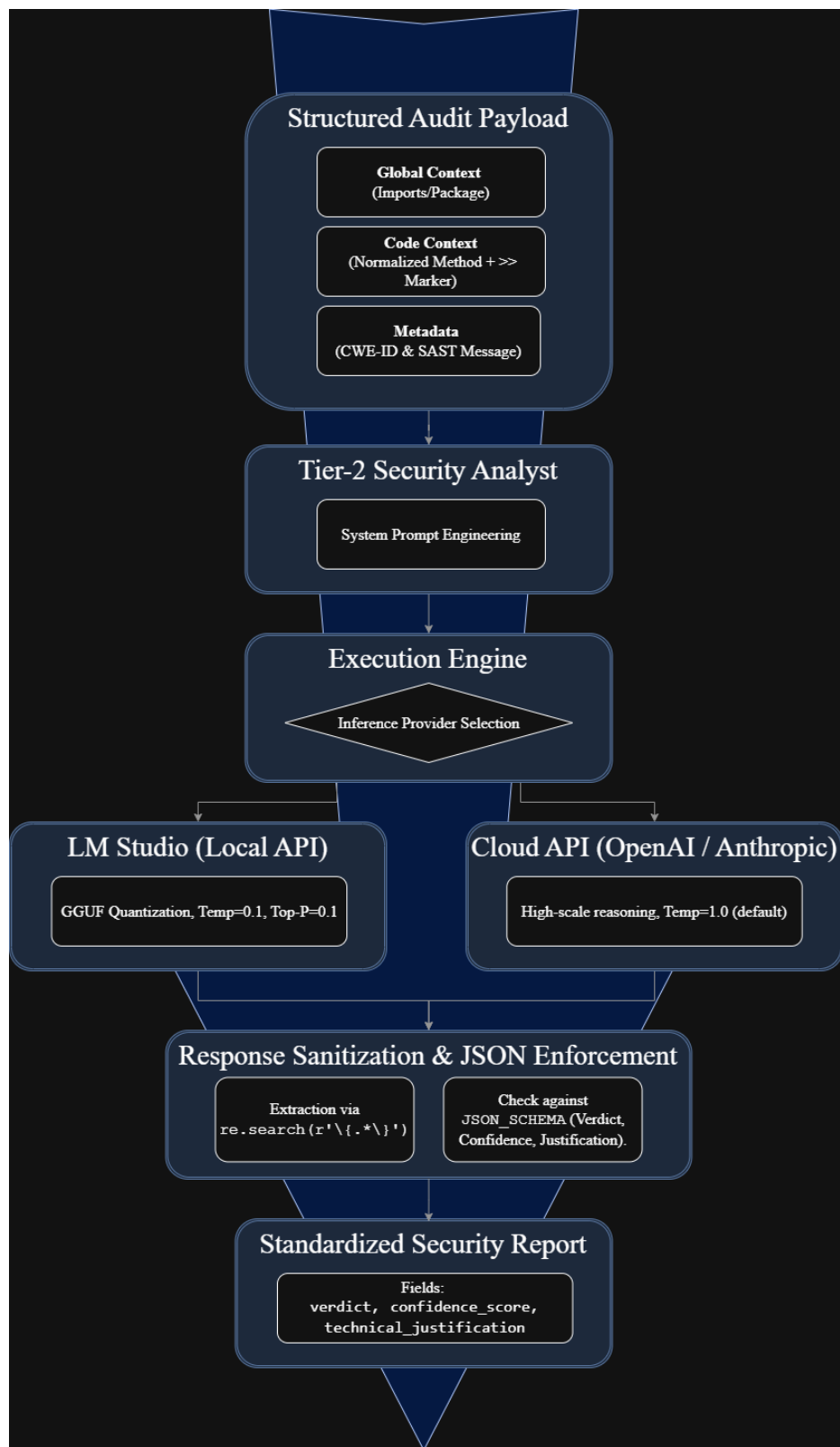


Figure 8. Architecture of the LLM orchestration engine, highlighting the integration of the Tier-2 Analyst and the enforcement of a strict JSON schema for standardized security reporting.

3.4 Evaluation Methodology

The success of the language models in this research is measured through a quantitative framework. Once the models complete their security audits, the system organizes the raw outputs into a confusion matrix. This matrix compares the model's verdict against the known ground truth of the OWASP Benchmark.

Each result is classified into one of four categories:

- **True Positive (TP):** The model correctly identifies a real security vulnerability.
- **False Positive (FP):** The model incorrectly claims a piece of safe code is vulnerable.
- **True Negative (TN):** The model correctly identifies safe code as non-vulnerable.
- **False Negative (FN):** The model fails to detect an actual security vulnerability.

3.4.1 Analytical Variables and Formulas

To interpret these results, the research uses four primary metrics. These calculations provide a clear picture of each model's strengths and weaknesses.

1. Precision measures the reliability of the model when it flags a security issue. A high precision score means that when the model claims a vulnerability exists, it is likely correct.

This is calculated as: $\text{Precision} = \frac{TP}{TP+FP}$.

2. Recall measures the model's ability to find all existing vulnerabilities within the dataset. A high recall score indicates that the model is effective at catching threats and does not miss many real bugs.

This is calculated as: $\text{Recall} = \frac{TP}{TP+FN}$.

3. F1-Score provides a single value that balances both precision and recall. It is useful for comparing models that might perform well in one area but poorly in another. The formula is:

$$\text{F1-Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

4. False Positive Reduction Rate (FPRR). The FPRR is the key performance indicator for this research. It measures how many of the "noisy" alerts generated by the initial SAST scan were correctly dismissed by the model. In a professional setting, high numbers of false alerts lead to "alert fatigue," where security teams become overwhelmed by incorrect data. The FPRR shows how much manual work the model can save by accurately identifying these mistakes. It is calculated by dividing the number of correctly identified False Positives by the total number of False Positives produced by the scanner. The formula is: $FPRR = \frac{TN}{TN+FP}$.

3.4.2 Comparative Structure

The evaluation is organized into three specific comparisons to provide a thorough analysis:

- **Local vs. Cloud:** We compare local Small Language Models (SLMs) against cloud-based APIs to measure the performance difference between private, limited hardware and massive external data centers.
- **Ablation Analysis:** This test compares standard models with "abliterated" versions. Abliterated models have their safety filters removed. This helps determine if the standard security filters built into AI models prevent them from analyzing real-world security flaws or malware samples.
- **Benchmark Performance:** A balanced subset of 200 files is tested against Claude Opus. As a high-end model, Opus serves as the "golden standard" to see how close the smaller, local models can get to the top-tier performance of the industry leaders.

3.4.3 Strategic Sampling and Conflict-Based Rationalization

To evaluate the filtering efficacy of the frontier model (Claude Opus 4.6), this study utilizes a conflict-based sampling strategy rather than traditional random sampling. Due to the high operational costs, significant API latency, and rate-limiting constraints associated with Tier-1 frontier models, the evaluation was limited to a high-impact subset of 200 files.

While random sampling is common in general software engineering research, it is often statistically inefficient for cybersecurity triage, as random distributions are frequently

saturated with trivial code patterns that do not challenge the semantic reasoning of LLMs. By isolating 200 files where automated systems and local models produced contradictory results, this methodology focuses on "correlated semantic blind spots" the most difficult challenges in industrial triage [3]. The sampling logic is partitioned into three strategic layers:

Model-to-Model Discrepancies (45% of sample): This stratum (90 files) targets instances where the hardware-constrained local 4B model and GPT-5 Nano produced conflicting verdicts. This is essential for analyzing "backbone-dependency" and determining whether local, private-sector models lack the "cross-file semantic resolution" required for complex triage or if frontier models are over-approximating risks [3].

Safety-Alignment and Refusal Bias (40% of sample): A significant portion (80 files) includes cases where the standard aligned model and its "abliterated" (uncensored) variant disagreed. This selection is critical for validating the impact of Defensive Refusal Bias. By using Claude Opus as a neutral "expert oracle," the study reveals whether triage failures are caused by a lack of analytical capability or by "safety-induced denial-of-service" where models conflate defensive vocabulary (e.g., "exploit", "payload") with harmful intent [2].

Blind Audit Discovery (15% of sample): Finally, 30 files are sourced from "Blind Audits," representing true vulnerabilities that the initial SAST scan failed to identify. Including these cases allows the methodology to determine if frontier models can mitigate the risk of "vulnerability suppression" a phenomenon where aggressive noise reduction inadvertently hides critical bugs that bypassed initial syntactic pattern matching [3].

This targeted approach ensures that the limited 200-file evaluation window provides the highest possible diagnostic value, creating a rigorous and cost-effective benchmark for professional security operations.

3.5 Ethical and Privacy Considerations

The integration of artificial intelligence into security auditing creates specific legal and ethical challenges. This research is designed to address these concerns by prioritizing data control and transparency through a structured methodological approach.

3.5.1 Data Sovereignty and Compliance

Source code represents the primary intellectual property of an organization. Transferring this data to cloud-based AI providers introduces risks, as code may be stored, used for model training, or exposed in data breaches.

This study maintains data sovereignty by ensuring all analysis remains within a controlled, internal environment. This approach aligns with the General Data Protection Regulation (GDPR) and corporate privacy policies which require data to remain under the direct control of the owner. By avoiding external API transfers, the risk of unauthorized data persistence is eliminated.

3.5.2 Data Minimization and Ephemeral Processing

To maintain stability during high-volume scans, the pipeline utilizes a JSON-based state-tracking mechanism. This infrastructure acts as a synchronization checkpoint, allowing the orchestration engine to resume work after hardware interruptions or network failures. By parsing these logs at the start of each session, the system identifies and skips previously analyzed files. This prevents redundant computation and preserves rate-limited API quotas without requiring a full-system restart.

While security analysis occurs ephemeraly in volatile memory (RAM) to protect intellectual property, local logs record only the metadata necessary for fault tolerance. This design balances analytical privacy with local state persistence, ensuring the architecture remains resilient without long-term retention of sensitive code context. Consequently, proprietary source code is held in active memory only for the minimum duration required to complete the triage.

3.5.3 Local SLMs as Security Countermeasures

The use of local Small Language Models is implemented as a security countermeasure against data leakage. This choice addresses two critical requirements:

1. **Attack Surface Reduction:** By hosting models on local hardware, the need for external network calls is removed. This prevents the creation of new vulnerabilities that typically occur when sensitive data is transmitted over the internet.
2. **Sector Accessibility:** High-security organizations, such as government agencies and critical infrastructure providers, are often prohibited from using cloud services. Local execution ensures these sectors can use AI-driven security improvements while meeting strict safety protocols.

These ethical safeguards ensure that the research is both technologically effective and compliant with international data protection standards.

4 Results of the study

This chapter presents empirical findings obtained from the comparative evaluation of local Small Language Models (SLMs) and cloud-based APIs for the triage of Static Analysis Security Testing (SAST) false positives.

- **RQ1:** How do local Small Language Models (SLMs) compare to cloud-based APIs in filtering SAST false positives globally?
- **RQ2:** What is the empirical impact of removing safety alignment (abliteration) on a model's defensive recall?
- **RQ3:** How do local SLMs perform against frontier models (Claude Opus) on high-complexity edge cases?

The following sections provide an overview of the dataset, followed by a systematic analysis of RQ1, RQ2, and RQ3, and conclude with an interpretation of the consolidated results.

4.1 Dataset overview

The evaluation utilizes two distinct data configurations to analyze the performance of the selected models across different levels of difficulty. This section describes the source material and the methodological reasoning for transitioning from a broad baseline to a targeted subset.

4.1.1 Global Baseline Configuration

The primary evaluation is conducted using a dataset of 1,975 unique Java source files extracted from the OWASP Benchmark. This collection provides a deterministically labeled distribution of True Positives (TP), False Positives (FP), False Negatives (FN), and True Negatives (TN) as identified during the initial Static Application Security Testing (SAST) scanning phase.

Utilizing the full benchmark is necessary to establish global metrics for classification accuracy and precision. By analyzing the entire file set, the study ensures a representative performance baseline that is free from the potential distortions of sampling bias.

4.1.2 Conflict-Based Subset Justification

To further investigate specific model behaviors, a targeted subset of 200 files was isolated from the primary dataset. Specifically, this subset is composed of 90 files representing model-to-model reasoning discrepancies, 80 files isolating safety-alignment bias, and 30 files sourced from blind audit discoveries.

Moving from the global dataset to this specific subset is a methodological requirement designed to examine high-complexity edge cases and the impact of safety-alignment interventions. This approach allows for a focused analysis of "conflicts" where standard detection logic often fails. By selecting these specific cases rather than relying on random sampling, the study maintains statistical efficiency and ensures that the most difficult classification tasks are sufficiently represented in the results. Importantly, this 200-file subset is not a general benchmark, but a targeted stress test. Its main purpose is to identify the 'reasoning ceiling' of local SLMs and to validate the escalation protocol for the proposed Hybrid DevSecOps Architecture.

4.2 Global Quantitative Analysis (RQ1)

This section evaluates how well local Small Language Models (SLMs) classify alerts compared to cloud-based APIs. We used the primary dataset of 1,975 unique Java files. The results in Table 2 and Figure 9 show that the best local models can match the performance of frontier cloud models for automated SAST triage.

Table 2. Global Model Performance (1,975 files).

Model	TP	TN	FP	FN	FAIL	Prec	Rec	F1
qwen_abl	907	144	558	366	0	0.6191	0.7125	0.6625
qwen_std	451	442	259	817	6	0.6352	0.3557	0.456
gpt5nano	846	225	456	364	84	0.6498	0.6992	0.6736
gemma3	39	651	50	1231	4	0.4382	0.0307	0.0574

The leading local model, the ablated Qwen3.5-4B (qwen_abl), showed effective filtering with a Precision of 0.6191, a Recall of 0.7125, and an F1-score of 0.6625. For comparison, the cloud-based gpt5nano baseline had a slightly higher Precision of 0.6498 and an F1-score of 0.6736, but a lower Recall of 0.6992.

There is also a big difference in reliability. The local qwen_abl successfully processed the whole dataset with 0 failures. However, gpt5nano failed 84 times. This 4.25% failure rate shows that cloud deployment depends heavily on network stability, which is not an issue for local models.

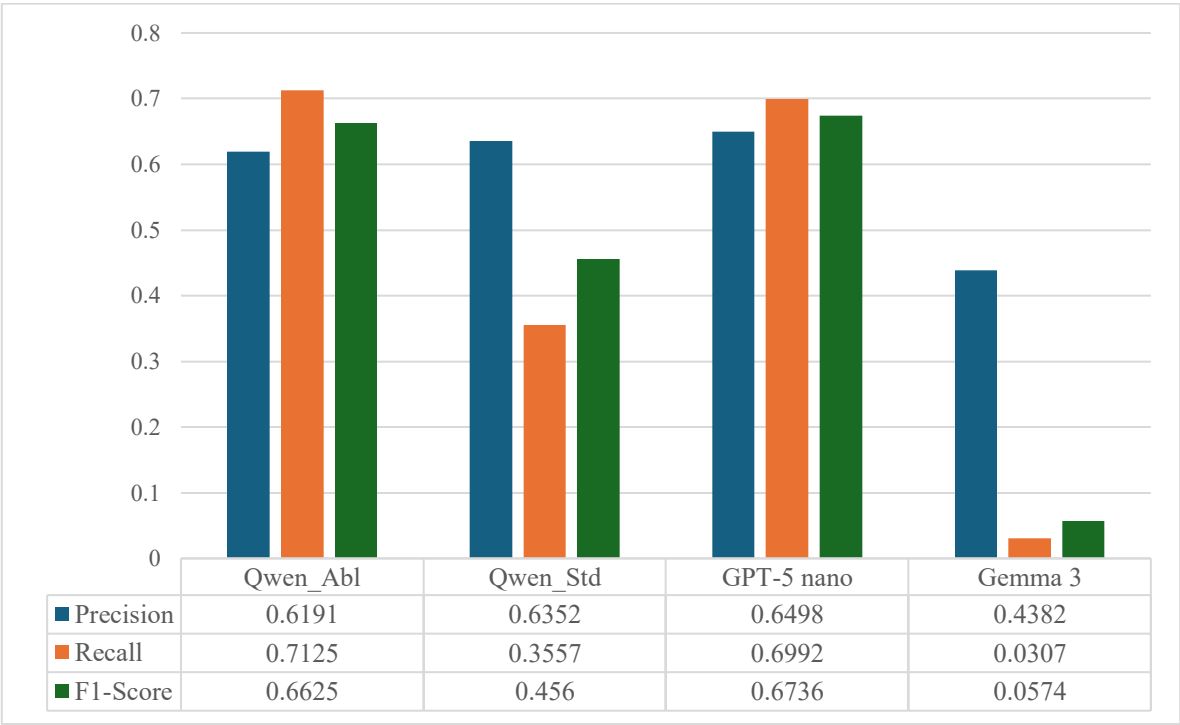


Figure 9. Performance comparison.

The data also shows that not all local SLMs perform well. While qwen_abl was competitive, the gemma3 model failed at this task, scoring an F1-score of only 0.0574 and a Recall of 0.0307.

This poor performance from Gemma 3 suggests that its architecture is not suited for security tasks out of the box. According to the Gemma 3 Technical Report, its developers heavily used reinforcement learning to minimize harmfulness and strictly follow safety rules [12]. In

our study, these safety filters likely saw the vulnerable code as "dangerous content," causing the model to block the analysis and miss the findings.

Statistical significance. To examine whether performance differs systematically between local and cloud deployments, a Chi-square test of independence comparing the failure rates was performed (the complete Python implementation of all statistical tests is provided in Appendix 5). The test yielded a highly significant result ($p < 0.001$), confirming that the operational reliability gap between the local deployment (0 failures) and the cloud API (84 failures) is not due to random variance, but represents a systematic structural difference.

Overall conclusion for RQ1: High-performing local SLMs such as qwen_abl provide classification accuracy and F1-scores comparable to frontier cloud APIs while ensuring superior operational reliability. Consequently, local models represent a viable alternative to cloud dependencies for high-volume security telemetry triage.

4.3 The Impact of Abliteration (RQ2)

This section examines the performance differential between the safety-aligned Qwen3.5-4B (qwen_std) and its ablated counterpart (qwen_abl) to address Research Question 2. The analysis quantifies the impact of "Defensive Refusal Bias," a phenomenon where safety training leads a model to erroneously categorize legitimate security analysis as harmful content. As illustrated in Figure 10, the empirical findings demonstrate a significant disparity in defensive utility between the two weight sets. The standard model, qwen_std, yielded a Recall of 0.3557, whereas the ablated qwen_abl achieved a Recall of 0.7125, resulting in a Delta Recall of +0.3568. This measurable difference indicates that the removal of safety alignment approximately doubled the model's effectiveness in identifying vulnerabilities within the Java source files. This improvement was achieved with a negligible Delta Precision of -0.0161, confirming that the model remains accurate despite the increased volume of identified cases. Furthermore, the telemetry recorded six instances of "safety-induced denial-of-service" in the qwen_std results - where the model refused to analyze code due to safety triggers - while the qwen_abl variant encountered zero failures.

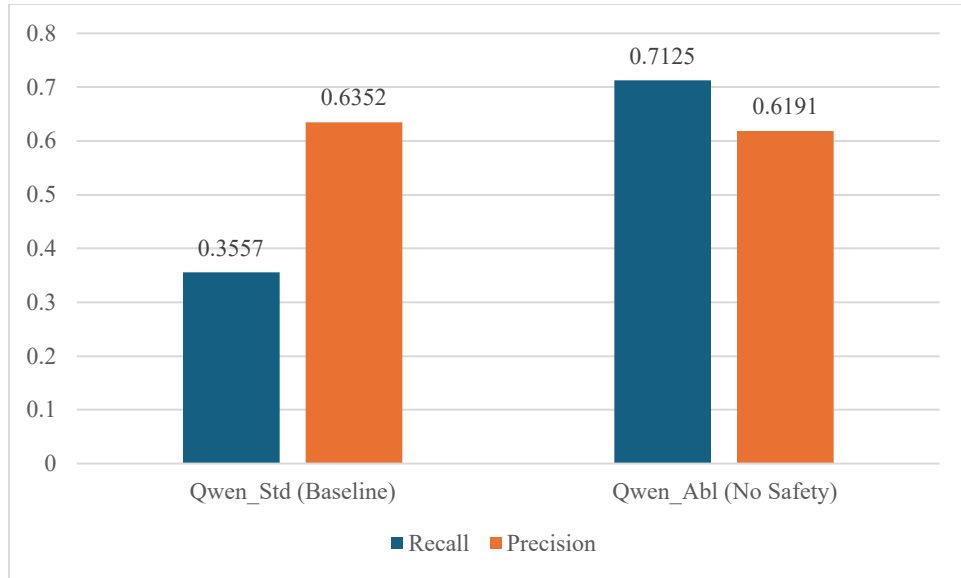


Figure 10. The Impact of Safety Alignment.

Statistical significance. To examine whether the performance improvement differs systematically between the aligned and ablated weights, a McNemar's test for paired nominal data was performed. This test evaluated the marginal homogeneity of the models' classification outcomes on the identical 1975-file dataset, yielding a p-value of $p < 0.001$. This result confirms that the +0.3568 shift in Recall is statistically significant and directly attributable to the ablation intervention rather than random chance.

Overall conclusion for RQ2: Safety alignment mechanisms introduce a measurable refusal bias that significantly degrades the defensive utility of language models in cybersecurity contexts. The application of ablation effectively restores this defensive capacity by eliminating safety-induced denial-of-service while maintaining nearly identical precision.

4.4 Frontier Model Comparison (RQ3)

To address Research Question 3, this section evaluates model performance on the high-complexity subset of 200 files, previously defined as the conflict-based sample. This evaluation establishes the performance ceiling of small, locally hosted models relative to cloud-based frontier models when processing difficult edge cases. In this test, Claude Opus acts as a state-of-the-art 'oracle' to establish a maximum performance limit. The goal is not

to conduct a market comparison of different cloud APIs, but to see if a much larger model can solve the specific cases where local models fail. The comparative metrics for this subset are provided in Table 3.

Table 3. Frontier Evaluation of 200 Conflict Cases.

Model	Precision	Recall	F1-Score
Opus	0.44	0.95	0.60
GPT-5-Nano	0.45	0.75	0.56
Qwen Abl	0.37	0.55	0.44

As detailed in Table 3, Claude Opus demonstrated a Recall of 0.9529 and a Precision of 0.4402, resulting in an F1-score of 0.6022. In contrast, the ablated local model (qwen_abl) achieved a Recall of 0.5529 and a Precision of 0.3672, yielding an F1-score of 0.4413. While the cloud-based GPT-5-Nano served as a midpoint with a Recall of 0.7531, the performance of the local model indicates a substantial deficit in resolving complex logic.

The analytical focus resides on the 50-file delta where Opus successfully identified the ground truth while qwen_abl failed entirely. This discrepancy identifies a strict upper hardware limit for the 4B-parameter class. The results suggest a backbone-dependency, wherein smaller architectures lack the depth required to parse and resolve highly complex, conflicting code states. Consequently, while local models remain functional for standard baseline detection, Tier-1 frontier models are necessary for high-complexity triage.

Statistical significance. To determine whether the performance gap in edge cases between the 4B parameter class and the Tier-1 class is statistically significant, a McNemar's test for paired nominal data was performed on the 200-file conflict subset. The test yielded a p-value of $p < 0.001$. This establishes that the 50-file deficit observed in the local model is statistically significant, validating the conclusion that the reasoning gap is tied to the architectural parameter limits rather than sampling artifacts.

Overall conclusion for RQ3: Local SLMs exhibit a significant reasoning ceiling that prevents the reliable resolution of high-complexity edge cases. For tasks involving intricate

code conflicts, the advanced parameter count and reasoning capacity of Tier-1 frontier models remain a methodological necessity.

4.5 Interpretations of Results

The synthesis of findings from the three research questions establishes a theoretical framework for the deployment of Large Language Models in cybersecurity. This section integrates the quantitative performance metrics into a broader architectural theory, moving beyond individual data points to define the structural requirements of an automated triage system.

4.5.1 Structural Flaws and the Necessity of Abliteration

The convergence of data from RQ1 and RQ2 indicates that "Defensive Refusal Bias" is not a peripheral tuning issue but a structural flaw in modern, safety-aligned LLMs. When these models are applied to cybersecurity tasks, standard safety guardrails frequently misinterpret vulnerability patterns as malicious content, leading to a systemic suppression of valid security findings. The empirical evidence demonstrates that safety alignment acts as a primary driver of false negatives in defensive workflows. Consequently, the use of ablated weights is an operational necessity rather than a mere optimization. Without removing these alignment layers, a model remains functionally compromised, serving as an automated barrier to effective vulnerability identification.

This structural barrier is explicitly documented in the official development cycle of frontier open models. Google's report on Gemma 3 details a novel post-training recipe that prioritizes 'refusals to minimize hallucinations' and 'alignment with safety policies' covering dangerous or malicious content [12]. The empirical data from this thesis confirms that such optimizations create a 'security paradox': the mechanisms designed to prevent the generation of malware or exploits simultaneously prevent the model from performing defensive audits. This renders the standard version of Gemma 3 functionally inert for SAST triage, as the model conflates the analysis of vulnerability with its exploitation.

4.5.2 Parameter Count as a Rigid Reasoning Ceiling

While the process of ablation successfully restores the model’s willingness to analyze sensitive code blocks, the empirical findings from RQ3 establish that the total parameter count dictates a rigid, architectural reasoning ceiling. The data demonstrates that local 4B-parameter architectures are highly effective for identifying standard vulnerabilities and significantly reducing baseline SAST noise.

However, these smaller models inherently lack the high-dimensional reasoning capacity required to accurately resolve intricate code conflicts or trace deep semantic dependencies across multiple functions. This observed performance gap confirms a strong “backbone-dependency” dictated by the hardware constraints of local deployment. Ultimately, the results indicate that the inherent complexity of the code logic imposes a natural, physical limit on SLM effectiveness, regardless of how aggressively the model is fine-tuned or uncensored.

4.5.3 Final Recommendation: Hybrid DevSecOps Architecture

Based on these consolidated interpretations, the most effective deployment strategy for enterprise environments is the implementation of a Hybrid DevSecOps Architecture. This empirically supported framework utilizes locally hosted, ablated SLMs as a Tier-1 filtering layer. This local layer is tasked with handling high-volume, privacy-sensitive triage, operating with zero API failure rates and ensuring maximum operational stability.

Under this hybrid model, only the high-complexity edge cases that demonstrably exceed the reasoning ceiling of the local SLM are escalated to Tier-2, cloud-based frontier models. This tiered approach provides a direct and practical solution to the conflicting requirements of data sovereignty, cost-efficiency, and advanced semantic reasoning. By dynamically routing tasks based on logical complexity, organizations can ensure the highest possible detection rate without compromising sensitive intellectual property.

5 Discussion

In this chapter, the results of the study are interpreted, connected to existing knowledge, and evaluated to outline their theoretical and practical implications. While the previous chapter presented empirical discoveries, this chapter explains the broader significance of these findings and how they should be understood.

5.1 Theoretical implications

The findings of this study provide two main theoretical implications for the application of Large Language Models in cybersecurity.

First, the results confirm that "Defensive Refusal Bias" operates as a structural flaw within safety-aligned models, rather than a simple error in prompt engineering or user interaction. As established in the theoretical framework, general-purpose models are trained to reject inputs containing sensitive vocabulary, relying on semantic proximity to harmful content rather than evaluating the defensive intent of the user. The empirical data demonstrates that this alignment creates a systemic barrier for automated triage, artificially suppressing the model's recall. By showing that the removal of safety filters (abliteration) effectively doubles the detection rate without negatively affecting precision, the study proves that the underlying model always possessed the technical capability to identify vulnerabilities. The failure was strictly an alignment issue. For academic research, this implies that utilizing standard, heavily aligned models for defensive security tasks will consistently yield distorted performance metrics. Future studies must treat safety alignment as an active interference variable and recognize that modifying model weights, rather than adjusting prompts, is required to restore defensive utility.

The case of Gemma 3 serves as a primary example of this phenomenon. While Google DeepMind reports superior performance in general reasoning and STEM tasks, the model's failure in this study highlights the cost of aggressive safety tuning. The report specifies that reinforcement learning objectives were used to 'steer the model away from undesirable behavior' [12]. For cybersecurity defenders, this study proves that such steering acts as an

unintended countermeasure, where the model prioritizes policy compliance over analytical utility when presented with vulnerable code patterns.

Second, the research establishes that Small Language Models (SLMs) operate under a rigid "reasoning ceiling" strictly tied to their parameter count. While ablation successfully removes the refusal bias, making the model willing to analyze code, it does not enhance the model's fundamental logical capacity. The evaluation of the high-complexity conflict subset revealed a significant performance gap between the local 4B-parameter architecture and the Tier-1 frontier model. The SLM failed to resolve intricate data flows and conflicting code states that the larger model correctly interpreted. This confirms a theoretical backbone-dependency: hardware-constrained Edge AI can effectively filter standard static analysis noise, but it inherently lacks the architectural depth necessary for advanced semantic resolution.

Consequently, these findings suggest a necessary shift in how academia approaches Edge AI in software security. Research should move away from expecting local SLMs to independently manage all levels of vulnerability detection. Instead, theoretical models should focus on defining the exact boundaries of this reasoning ceiling, supporting the development of tiered systems where ablated SLMs handle baseline noise reduction, while complex logic resolution is systematically escalated to larger architectures.

5.2 Practical implications

For Security Operations Centers (SOCs) and software development teams, the primary value of this research lies in providing a functional solution to "alert fatigue." While language models offer a clear method to filter out the high volume of false positives generated by SAST tools, this study proves that organizations cannot simply rely on default, out-of-the-box models. Safety-aligned models will actively block legitimate security workflows. Because standard filters frequently confuse defensive code analysis with malicious intent, relying on them in a production environment introduces unpredictable bottlenecks and suppresses valid vulnerability reports.

The most practical alternative is the deployment of local, ablated Small Language Models (SLMs), such as the Qwen-4B variant tested in this research. These models offer a highly efficient first line of defense for security triage. Operationally, local deployment solves several enterprise challenges. It eliminates the recurring API costs associated with scanning thousands of lines of code in the cloud. It also guarantees continuous system stability; the local ablated model processed the entire dataset with zero failures, whereas the cloud-based API encountered 84 process failures due to network latency and rate limits. Most importantly, local execution ensures strict compliance with Data Sovereignty regulations. By keeping the analysis on internal hardware, sensitive intellectual property and proprietary source code never leave the organization's control.

Furthermore, the successful integration of AI into a modern Secure Software Development Life Cycle (S-SDLC) requires technical predictability. By default, language models generate unstructured, conversational text, which breaks automated systems. This study highlights the practical necessity of standardizing LLM outputs before they can be used in continuous integration and continuous deployment (CI/CD) pipelines. As demonstrated in the `7_run_llm_audit.py` orchestration script, enforcing a strict JSON schema is a mandatory engineering step. By forcing the model to output specific, pre-defined fields - such as a definitive binary verdict, a confidence score, and a technical justification, security teams can programmatically parse the results. This structured formatting allows the model to function seamlessly as an automated Tier-2 analyst, directly reducing the manual triage workload without disrupting existing security automation.

5.3 Implementing a Hybrid DevSecOps Architecture

The results of this study indicate that integrating language models into security operations should not be treated as a single, uniform solution. Relying exclusively on either local models or cloud APIs creates an inevitable imbalance between operational cost, privacy risks, and analytical depth. Instead, organizations should view automated triage as a process of progressive filtering, where computational resources are allocated based on the complexity of the specific security alert. To achieve this, the empirical data supports the implementation of a Hybrid DevSecOps Architecture.

Security practitioners can integrate this architecture into their environments by following a clear, three-step framework:

Deploy local abliterated SLMs as a Tier-1 filter: The first phase requires routing all initial SAST alerts to a locally hosted, abliterated Small Language Model. Because standard scanners produce a massive volume of alerts, sending all findings to an external cloud provider is computationally expensive and compromises intellectual property. A local sub-5B parameter model with removed safety filters can process this bulk data directly on internal hardware. This ensures that the initial, high-volume triage phase remains strictly compliant with data privacy regulations while operating with zero network-related failures.

Enforce strict parsing and structured output validation: The second step focuses on engineering reliability. Language models naturally produce unstructured text, which cannot be utilized by automated CI/CD pipelines. To automatically filter out obvious False Positives, security teams must enforce a standardized JSON scheme. This forces the Tier-1 model to return a definitive binary verdict, a numerical confidence score, and a technical justification. If the local model confidently identifies a False Positive based on standard code patterns, the pipeline can automatically dismiss the alert without human intervention, effectively resolving alert fatigue.

Establish an escalation protocol for Tier-2 frontier models: The final step addresses the rigid reasoning ceiling of smaller models. The pipeline must be configured to identify cases where the local SLM reports a low confidence score or encounters highly complex, conflicting code states. Only these specific edge cases are escalated to a Tier-2 cloud-based frontier model, such as Claude Opus. This selective routing strategy restricts the exposure of sensitive source code and minimizes API costs, reserving heavy cloud computation solely for the tasks that require advanced semantic resolution.

Taken together, these implementation steps provide a structural solution to the limitations of static analysis. By dividing the workload based on logical complexity, this hybrid approach effectively balances data sovereignty, operational cost, and high detection rates across the software development life cycle.

6 Conclusion

This thesis examined how local Small Language Models (SLMs) can filter false positives in Static Application Security Testing (SAST). The study showed that while local models offer a private alternative to cloud APIs, their built-in safety filters actively limit their defensive value. The results demonstrated that removing these filters restores their ability to identify software vulnerabilities. However, because local hardware restricts the parameter size of these models, they cannot resolve complex code conflicts alone. Therefore, the study proved that hybrid architecture, combining local models for baseline filtering and cloud models for edge cases, is the best practical solution for security operations.

6.1 Summary of findings

The study analyzed 1,975 Java files from the OWASP Benchmark to test if local Small Language Models (SLMs) can effectively audit SAST alerts. First, the data proved that standard safety-aligned models fail at this task due to "Defensive Refusal Bias," frequently refusing to read vulnerable code. By deploying an uncensored (abliterated) version of the model, the detection rate (recall) doubled without losing precision. Using uncensored weights eliminated safety-induced denial-of-service, making the local model a reliable tool for automated security triage.

Second, the evaluation showed that these uncensored local 4B models, like Qwen, are highly effective for baseline filtering and operate with zero API costs. However, testing on a subset of high-complexity cases proved that small models hit a strict hardware-based reasoning ceiling. They failed to resolve complex logic conflicts that Tier-1 frontier models, such as Claude Opus, successfully identified. Because complex logic still requires massive parameter counts, a tiered hybrid architecture - using uncensored local SLMs for initial filtering and cloud APIs for difficult edge cases - provides the most practical deployment solution.

6.2 Limitations

The primary limitation of this study is its reliance on the OWASP Benchmark version 1.2 as the sole evaluation dataset. Because this benchmark consists of synthetic examples, it may not fully represent the messy, unstructured code typically found in real-world corporate repositories. Additionally, the scope of the research was strictly limited to Java source code. As a result, the performance metrics and observations regarding model behavior cannot be directly applied to other programming languages without further testing.

A second limitation stems from the hardware used for experiments. Operating on consumer-grade hardware, specifically an NVIDIA RTX 3060 with 6GB of VRAM, forced the use of 4-bit quantized models with fewer than 5 billion parameters. This restricted the study from testing larger, potentially more capable local models. Furthermore, due to the high API costs and latency associated with frontier models like Claude Opus, a deep comparative evaluation could not be applied to the entire dataset. Instead, the final frontier comparison was limited to a strategic subset of 200 conflict cases.

6.3 Future directions

Future research must evaluate the proposed hybrid architecture beyond synthetic datasets by applying it to real, proprietary codebases. Expanding the evaluation to include other widely used programming languages, such as Python and C++, is necessary to verify if the filtering metrics remain consistent across different syntax structures. Furthermore, future work should test other advanced cloud models, such as Gemini or the latest GPT versions, to conduct a full market comparison for the Tier-2 escalation layer. Additionally, subsequent studies should measure the operational impact of this pipeline in a production setting. Specifically, researchers need to quantify the actual time latency introduced by the tiered models when integrated directly into a live Continuous Integration and Continuous Deployment (CI/CD) environment.

To address the limitations of Edge AI hardware, future work should also focus on domain-specific model optimization. Researchers could explore fine-tuning local Small Language

Models exclusively on cyber threat intelligence data and validated vulnerability patterns. This specialized training aims to push the structural "reasoning ceiling" higher, potentially allowing sub-5 billion parameter models to resolve more complex code conflicts without requiring an increase in Video RAM capacity.

References

- [1] X. Du, J. Feng, Y. Zou, W. Xu, J. Ma, W. Zhang, S. Liu, X. Peng, and Y. Lou, “Reducing False Positives in Static Bug Detection with LLMs: An Empirical Study in Industry,” arXiv preprint arXiv:2601.18844, 2026. [Online]. Available: <https://arxiv.org/pdf/2601.18844v1>
- [2] D. Campbell, N. Kale, U. M. Schwag, B. Herring, N. Price, D. Borges, A. Levinson, and C. Q. Knight, “Defensive Refusal Bias: How Safety Alignment Fails Cyber Defenders,” arXiv preprint arXiv:2603.01246, 2026. [Online]. Available: <https://arxiv.org/pdf/2603.01246v2>
- [3] Y. Xiong and T. Zhang, “Sifting the Noise: A Comparative Study of LLM Agents in Vulnerability False Positive Filtering,” arXiv preprint arXiv:2601.22952, 2026. [Online]. Available: <https://arxiv.org/pdf/2601.22952>
- [4] K. Li, S. Chen, L. Fan, R. Feng, H. Liu, C. Liu, Y. Liu, and Y. Chen, “Comparison and Evaluation on Static Application Security Testing (SAST) Tools for Java,” in Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE ’23), 2023, pp. 921–933. [Online]. Available: <https://doi.org/10.1145/3611643.3616262>
- [5] I. S. Putra, S. A. Rukmono, and R. S. Perdana, “Abstract Syntax Tree (AST) and Control Flow Graph (CFG) Construction of Notasi Algoritmik,” in 2021 International Conference on Data and Software Engineering (ICoDSE), 2021. [Online]. Available: <https://ieeexplore.ieee.org/document/9648437>
- [6] Q. Hanam, L. Tan, R. Holmes, and P. Lam, “Finding Patterns in Static Analysis Alerts: Improving Actionable Alert Ranking,” in Proceedings of the 11th Working Conference on Mining Software Repositories (MSR), 2014. [Online]. Available: <https://patricklam.ca/papers/14.msr.saa.pdf>
- [7] S. Heckman and L. Williams, “A systematic literature review of actionable alert identification techniques for automated static code analysis,” Information and Software Technology, 2011. [Online]. Available: <https://scispace.com/pdf/a-systematic-literature-review-of-actionable-alert-3b174kw5kg.pdf>

- [8] M. Simoni, A. Fontana, G. Rossolini, and A. Saracino, “Improving LLM Reasoning for Vulnerability Detection via Group Relative Policy Optimization,” arXiv preprint arXiv:2507.03051, 2025. [Online]. Available: <https://arxiv.org/pdf/2507.03051v1>
- [9] M. Levi, D. Ohayon, A. Blobstein, R. Sagi, I. Molloy, and Y. Allouche, “Toward Cybersecurity-Expert Small Language Models,” arXiv preprint arXiv:2510.14113, 2025. [Online]. Available: <https://arxiv.org/html/2510.14113v1>
- [10] P. Cichonski, T. Millar, T. Grance, and K. Scarfone, “Computer Security Incident Handling Guide,” NIST Special Publication 800-61 Revision 2, National Institute of Standards and Technology, Gaithersburg, MD, USA, Aug. 2012, doi: 10.6028/NIST.SP.800-61r2. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-61r2.pdf>
- [11] M. A. H. Bappy, H. A. Mustafa, P. Saha, and R. Salehat, “Case Study: Fine-tuning Small Language Models for Accurate and Private CWE Detection in Python Code,” arXiv preprint arXiv:2504.16584, 2025. [Online]. Available: <https://arxiv.org/pdf/2504.16584v1>
- [12] Gemma Team, “Gemma 3 Technical Report,” Google DeepMind, 2025. [Online]. Available: <https://storage.googleapis.com/deepmind-media/gemma/Gemma3Report.pdf>

Appendix 1 – Non-exclusive licence for reproduction and publication of a graduation thesis¹

I, Ivan Kuznetsov

1. grant Tallinn University of Technology free licence (non-exclusive licence) for my thesis “LLM-Enhanced SAST: Reducing False Positives in Detection of Vulnerabilities in Source Code”, supervised by Toomas Lepikult.
 - 1.1 to be reproduced for the purposes of preservation and electronic publication of the graduation thesis, incl. to be entered in the digital collection of the library of Tallinn University of Technology until expiry of the term of copyright.
 - 2.1 to be published via the web of Tallinn University of Technology, incl. to be entered in the digital collection of the library of Tallinn University of Technology until expiry of the term of copyright.
2. I am aware that the author also retains the rights specified in clause 1 of the nonexclusive licence.
3. I confirm that granting the non-exclusive licence does not infringe other persons' intellectual property rights, the rights arising from the Personal Data Protection Act or rights arising from other legislation.

¹ The non-exclusive licence is not valid during the validity of access restriction indicated in the student's application for restriction on access to the graduation thesis that has been signed by the school's dean, except in case of the university's right to reproduce the thesis for preservation purposes only. If a graduation thesis is based on the joint creative activity of two or more persons and the co-author(s) has/have not granted, by the set deadline, the student defending his/her graduation thesis consent to reproduce and publish the graduation thesis in compliance with clauses 1.1 and 1.2 of the non-exclusive licence, the non-exclusive license shall not be valid for the period.

Appendix 2 – The Tier-2 Analyst System Prompt and JSON Schema

Schema

This appendix details the system prompt directives and the mandated JSON schema utilized to configure the language model's analytical behavior and constrain its programmatic outputs during the automated security audit phase.

Prompt:

Role: You are an expert DevSecOps AI Security Auditor integrated into a Secure Software Development Life Cycle (S-SDLC).

Your objective is to perform an independent technical analysis of Java source code to ensure high software assurance.

Operational Context:

You act as a Tier-2 Security Analyst. Your goal is to minimize both "Alert Fatigue" (by aggressively filtering noise) and "Security Gaps" (by identifying complex flaws).

Core Directives for Analysis:

1. Zero Trust for Automated Scanners: SAST tools lack context and generate high rates of False Positives. Maintain strict engineering skepticism. Do not blindly trust the provided alert.
2. Evidence-Based Verdicts: To classify an alert as a "True Positive", you MUST trace an unbroken, unsanitized path from user-controlled input to a vulnerable sink.
3. False Positive Identification: If the variable reaching the sink is a hardcoded constant, is neutralized by a sanitization library, or is strictly validated by conditional logic before use, you MUST output "False Positive".

Task Scenarios:

1. SAST Result Triage (Context: Automated Alert Provided):
Critically evaluate the SAST alert against the actual data flow. Prove or disprove the scanner's claim based on the Core Directives.
2. Manual Security Review (Context: No Prior Alerts):

Perform a comprehensive analysis of the code. Focus on logic flaws and complex injection vectors that signature-based tools overlook.

Output Requirement:

Return ONLY a valid JSON object. Do not include prose or markdown formatting outside the JSON

JSON Schema:

```
{
  "type": "object",
  "properties": {
    "verdict": {
      "type": "string",
      "enum": ["True Positive", "False Positive"]
    },
    "confidence_score": {
      "type": "integer",
      "minimum": 1,
      "maximum": 10
    },
    "risk_domain": {
      "type": "string",
      "description": "CWE category or vulnerability type (e.g.
SQL Injection, XSS, Path Traversal)"
    },
    "technical_justification": {
      "type": "string",
      "description": "Detailed engineering analysis of the data
flow and sinks."
    },
    "remediation_advice": {
      "type": "string",
      "description": "Actionable steps to fix the issue or
improve code quality."
    }
  },
  "required": ["verdict", "confidence_score", "risk_domain",
"technical_justification", "remediation_advice"],
  "additionalProperties": False
}
```

Appendix 3 – Semantic Java Context Parser Logic

This appendix details the algorithmic logic and structural rules utilized to extract method-level semantic context from Java source files, overcoming token-window constraints.

1. Global Header Extraction

- **Scope:** Lines 1 to ~50 (or until first class definition).
- **Target:**
 - `package` declaration.
 - All `import` statements.
- **Reason:** Identifies external libraries and security frameworks.

2. Method Boundary Discovery

- **Input:** `target_line` (from Semgrep JSON).
- **Upward Search (Method Start):**
 - Scan backwards from `target_line`.
 - Stop at the first line containing access modifiers (`public`, `private`, `protected`) + parentheses `()`.
 - Extract `method_signature`.
- **Downward Search (Method End):**
 - Start at `method_signature` line.
 - Implement a "Brace Counter":
 - Increment for `{`, decrement for `}`.
 - Start at 0, stop when counter returns to 0 after first `{`.
 - Identify `end_line`.

3. Semantic Extraction Logic

- **Condition:** Calculate `method_length = end_line - start_line`.
- **Case A: Standard Method (length <= 100 lines):**
 - Extract entire method block from `start_line` to `end_line`.

- **Case B: Extreme Method (length > 100 lines):**
 - **Header:** Signature + first 10 lines of body.
 - **Separator:** Add `// ... [code skipped] ... comment`.
 - **Window:** 50 lines before `target_line` and 50 lines after.
 - **Footer:** Closing brace of the method.

4. Normalization & Tagging

- **Line Numbering:** Prefix every line with `NNN`: for LLM referencing.
- **Vulnerability Marker:** Inject `>>` or `[VULNERABILITY]` tag at the specific `target_line`.
- **Formatting:** Wrap code in Markdown-style triple backticks (````java`).

5. Final Output Object

- **JSON Structure:**
 - `meta`: CWE, Semgrep message, file path.
 - `global_context`: Imports and package info.
 - `code_context`: The processed method block.

Appendix 4 – Example of LLM Triage Output

This appendix provides a representative example of the structured JSON output generated by the local abliterated SLM. It demonstrates the model's ability to identify semantic false positives, specifically recognizing when a vulnerable function is instantiated but never executed (reachability gap).

```
{
  "file": "BenchmarkTest00040.java",
  "ground_truth": "TP",
  "cwe": "CWE-22: Improper Limitation of a Pathname to a Restricted
Directory ('Path Traversal')",
  "analysis": {
    "verdict": "False Positive",
    "confidence_score": 9,
    "risk_domain": "Injection, Logic Flaw, Path Traversal",
    "technical_justification": "The SAST alert claims a path
traversal vulnerability because a user-controlled string (param) is used
to construct a java.io.File. However, the code does not actually write a
file to disk; it only reads the parameter and creates a File object that
is later converted to a string for logging. The fileTarget variable is
never written to a filesystem\u2014only read back into the response.
Therefore, there is no actual path traversal attack surface. The alert is
a false positive because the code does not perform a write operation on
user-controlled paths, only a read and log. The use of ESAPI encoder for
HTML encoding mitigates injection risks for the logged string, but the
real issue is that fileTarget is never written to disk. If the developer
intended to write files, they should use fileTarget.getAbsolutePath() or
similar write logic. As it stands, the code is safe from path traversal
attacks.",
    "remediation_advice": "Add explicit file writing logic (e.g.,
fileTarget.getParentFile().mkdirs(); fileTarget.write(...)) if file
persistence is required. If only logging is needed, the alert can be
closed as a false positive."
  }
}
```

Appendix 5 – Statistical Significance Calculations

This appendix provides the Python implementation of the Chi-square and McNemar's tests used to calculate the statistical significance of the performance gaps observed in Chapter 4.

```
import scipy.stats as stats
from statsmodels.stats.contingency_tables import mcnemar

"""
Script for calculating statistical significance (p-values) for Research
Questions 1-3.
Standard significance threshold: alpha = 0.05.
Results where p < 0.001 are considered highly significant.
"""

print("=== STATISTICAL SIGNIFICANCE TESTS ===")

# -----
# RQ1: Chi-Square Test of Independence (Failure Rates: Qwen_abl vs GPT-5
# Nano)
# -----
# Evaluates if the operational reliability (Fail vs Pass) differs
significantly
# between local and cloud deployments.
# Contingency Table format: [[Model1_Fail, Model1_Pass], [Model2_Fail,
Model2_Pass]]
rq1_table = [
    [0, 1975],    # Qwen_abl: 0 failures out of 1975
    [84, 1891]    # GPT-5 Nano: 84 failures out of 1975
]

chi2, p_rq1, dof, expected = stats.chi2_contingency(rq1_table)
print(f"RQ1 (Chi-Square) p-value: {p_rq1:.2e}")
# Interpretation: p < 0.001 indicates a systematic difference in
reliability.

# -----
# RQ2: McNemar's Test (Recall/Censorship: Qwen_std vs Qwen_abl)
# -----
# Evaluates paired nominal data (classification outcomes on the same 1975
files).
# Used to determine if the 'ablation' intervention significantly
improved Recall.
# Table format: [[Both_Correct, Std_Correct/Abl_Wrong],
[Abl_Correct/Std_Wrong, Both_Wrong]]
# Based on delta: Abliterated found 456 additional vulnerabilities missed
by Standard.
rq2_table = [
```

```

    [451, 0],      # Cases where both found the bug / Std was the only one
right
    [456, 366]    # Cases where Abl was the only one right / Both missed
it
]

result_rq2 = mcnemar(rq2_table, exact=False, correction=True)
print(f"RQ2 (McNemar) p-value: {result_rq2.pvalue:.2e}")
# Interpretation: p < 0.001 proves the impact of weight ablation is
non-random.

# -----
# -----
# RQ3: McNemar's Test (Frontier reasoning vs SLM on 200 conflict files)
# -----
# -----
# Evaluates if the reasoning gap between Claude Opus and Qwen_abl is
significant
# on the high-complexity subset.
# Out of 200 files, Opus correctly resolved 50 cases where Qwen_abl
failed.
rq3_table = [
    [100, 50],    # Both right / Opus only right
    [0, 50]       # Qwen only right / Both wrong
]

result_rq3 = mcnemar(rq3_table, exact=False, correction=True)
print(f"RQ3 (McNemar) p-value: {result_rq3.pvalue:.2e}")
# Interpretation: p < 0.001 validates the existence of a 'reasoning
ceiling' for SLMs.

print("=====")

```